

LAPORAN STUDI KASUS
STUDENT ACADEMIC PERFORMANCE TRACKER

Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny ST., M.AIT



Disusun oleh
Kelompok 8:

Rasikha Qarira Rohmad	(25051030025)
Muadi Ma'rufin	(25051030001)
Eksan Satya Purnama	(25051030024)
Faiq Ridha Prasetya	(25051030035)

PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026

BAB I

PENDAHULUAN

1.1. Latar Belakang

Sistem informasi akademik di tingkat universitas memiliki tantangan dalam mengelola data ribuan mahasiswa yang terus berkembang. Setiap semester, transkrip nilai mahasiswa perlu diperbarui secara berkala, divalidasi dengan prasyarat kurikulum, dan direkam dengan meminimalisasi kesalahan input. Pengelolaan data berskala besar ini menuntut efisiensi tinggi, baik dalam hal waktu eksekusi maupun penggunaan memori, yang tidak bisa dipecahkan hanya dengan struktur data dasar secara linear.

Oleh karena itu, diperlukan perancangan sistem pelacak performa akademik berbasis *Command Line Interface* (CLI) yang mengintegrasikan berbagai struktur data secara terpadu. Penggunaan *Binary Search Tree* (BST) diperlukan untuk pencarian data mahasiswa secara efisien. *Doubly Linked List* (DLL) dirancang untuk menyimpan transkripsi nilai yang fleksibel ditelusuri maju-mundur. Selain itu, graf terarah atau *Directed Acyclic Graph* (DAG) digunakan untuk memodelkan prasyarat matakuliah secara logis, sementara struktur *Stack* digunakan untuk menyimpan riwayat input guna mengantisipasi pembatalan operasi (*undo*) jika terjadi *human error*.

1.2 Tujuan Sistem

Tujuan utama dari pengembangan sistem *Student Academic Performance Tracker* ini adalah:

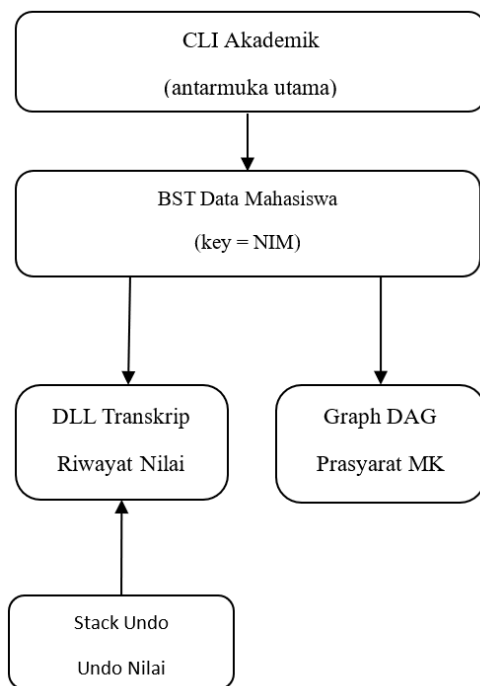
1. Merancang dan mengimplementasikan aplikasi CLI nyata secara mandiri untuk memonitor performa akademik mahasiswa secara interaktif.
2. Mengintegrasikan minimal empat struktur data secara fungsional (*Binary Search Tree*, *Doubly Linked List*, *Stack*, dan *Graph*) tanpa menggunakan pustaka (*library*) bawaan Python.
3. Menganalisis kompleksitas waktu (Big-O) dari setiap struktur data untuk melihat *trade-off* efisiensi pada pencarian mahasiswa, penyortiran IPK, dan pemrosesan transkrip.

1.3 Batas Implementasi

Agar ruang lingkup proyek tetap fokus, implementasi sistem ini dibatasi pada parameter berikut:

- Pengguna dan Entitas: Sistem mengelola data tetap berupa 60 mahasiswa (dengan format NIM awal 21XXXXXXX) dari 5 program studi (Teknik Elektro, Informatika, Mesin, Sipil, Kimia).
- Matakuliah dan Durasi: Sistem mencakup manajemen transkrip selama 8 semester dengan total ketersediaan 40 matakuliah.
- Penilaian: Sistem konversi nilai atau *grade* yang digunakan mencakup A (4.0), A- (3.7), B+ (3.3), B (3.0), B- (2.7), C+ (2.3), C (2.0), D (1.0), hingga E (0.0).
- Bahasa Pemrograman: Implementasi sistem sepenuhnya menggunakan Python 3. Pembuatan struktur data (Linked List, Stack, Tree, Graph) diwajibkan dibangun dari nol (secara manual) dan dilarang menggunakan *library* struktur data bawaan kecuali untuk *list* murni sebagai *array* dasar.

1.4 Diagram Arsitektur Sistem



Visualisasi Konsep Diagram:

1. CLI Akademik (Antarmuka Utama): Menerima input perintah pengguna (seperti CARI_MHS_INPUT_NILAI, TRANSKRIPSI dan memanggil fungsi pada struktur data yang relevan).
2. BST Data Mahasiswa: Berperan sebagai pusat basis data mahasiswa (Kunci = NIM).
3. DLL Transkrip Nilai: Terhubung di dalam setiap *node* BST. Setiap *node* mahasiswa memiliki satu antrean *Doubly Linked List* yang merekam riwayat nilainya.

4. Stack Undo: Modul eksternal yang memonitor operasi INPUT_NILAI. Jika UNDO_NILAI dipanggil, modul ini akan menghapus entri terakhir pada DLL milik NIM terkait.
5. Graph DAG Prasyarat: Modul eksternal independen yang diakses saat mahasiswa akan menginput nilai matakuliah baru. Modul ini memvalidasi apakah prasyarat (melalui *Topological Sort*) telah terpenuhi.

BAB II

PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data dan Justifikasi

Untuk memenuhi kebutuhan *Student Academic Performance Tracker*, sistem ini dibangun dengan mengintegrasikan empat struktur data utama. Pemilihan struktur data ini didasarkan pada karakteristik operasi yang dibutuhkan agar mendapatkan performa komputasi yang optimal:

1. Binary Search Tree (BST) untuk Data Mahasiswa

Justifikasi: Sistem membutuhkan fitur pencarian data mahasiswa secara sering dan cepat berdasarkan NIM. Penggunaan *array* biasa akan membutuhkan waktu pencarian $O(n)$, namun dengan BST (di mana kunci pengurutannya adalah NIM), operasi pencarian (CARI_MHS), penambahan, dan pembaruan IPK rata-rata dapat dilakukan dalam waktu $O(\log n)$.

2. Doubly Linked List (DLL) untuk Transkrip Nilai

Justifikasi: Setiap mahasiswa memiliki transkrip nilai yang dinamis. DLL dipilih karena mengizinkan operasi penambahan matakuliah di akhir (*insert tail*) dalam waktu $O(1)$. Selain itu, fitur pembatalan input nilai membutuhkan penghapusan nilai paling akhir (*remove tail*), yang pada DLL juga dapat dieksekusi dengan sangat efisien yaitu $O(1)$ tanpa perlu memindahkan elemen lain.

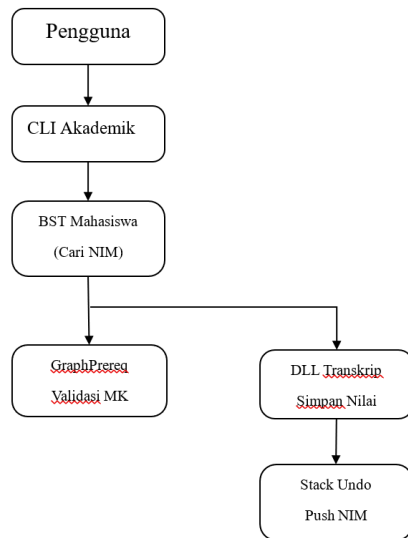
3. Stack untuk Mekanisme Undo

Justifikasi: Sistem dirancang untuk mampu membatalkan (*undo*) input nilai apabila terjadi kesalahan. *Stack* beroperasi dengan prinsip *Last-In-First-Out* (LIFO), yang sangat sempurna untuk merekam riwayat NIM mana yang terakhir kali melakukan input. Operasi *push* dan *pop* pada *Stack* masing-masing memakan waktu $O(1)$.

4. Directed Acyclic Graph (DAG) untuk Prasyarat Matakuliah

Justifikasi: Relasi prasyarat matakuliah tidak bersifat hierarkis murni maupun linear, melainkan berbentuk jaringan ketergantungan searah. Graf berarah tanpa siklus (DAG) sangat ideal untuk memodelkan sistem ini (Simpul = Matakuliah, Sisi = Relasi prasyarat). Melalui Graph DAG, sistem dapat memvalidasi syarat kelulusan dan menghasilkan urutan pengambilan matakuliah secara logis menggunakan algoritma *Topological Sort*.

2.2 Diagram Interaksi Antar Modul



Deskripsi Konseptual Diagram: Sistem berpusat pada Modul CLI sebagai pengontrol utama.

- Saat ada perintah input nilai, CLI akan memanggil BST untuk mencari data mahasiswa target.
- Sebelum menginput nilai, sistem memanggil GraphPrereq untuk memvalidasi apakah mahasiswa tersebut telah lulus matakuliah prasyarat.
- Jika lolos, nilai baru disisipkan ke DLL Transkrip Nilai milik mahasiswa tersebut.
- Setelah berhasil diinput, NIM mahasiswa dikirim (*push*) ke Stack Undo.
- Jika pengguna melakukan perintah UNDO, Stack Undo di-*pop* untuk mendapatkan NIM terakhir, lalu sistem memerintahkan DLL pada NIM tersebut untuk menghapus *node* terakhirnya (*remove tail*), dan BST memperbarui nilai IPK.

2.3 Pseudocode Algoritma Utama

Berikut adalah rancangan *pseudocode* untuk operasi kritis di dalam sistem:

1. Pseudocode Pembatalan Nilai (Undo Input) dengan Stack dan DLL

FUNCTION undo_nilai(global_stack, bst_mahasiswa):

IF global_stack is empty:

PRINT "Tidak ada operasi untuk dibatalkan"

RETURN

```
nim_terakhir = global_stack.pop() // Ambil NIM dari Stack (O(1))
node_mhs = bst_mahasiswa.search(nim_terakhir) // Cari di BST (O(log n))
```

```
IF node_mhs is not null:
    node_mhs.transkripsi.hapus_terakhir() // Hapus tail DLL (O(1))
    node_mhs.update_ipk() // Hitung ulang IPK di BST
    PRINT "Input nilai terakhir untuk NIM", nim_terakhir, "berhasil
dibatalkan"
```

2. Pseudocode Validasi Prasyarat Matakuliah (Graph DAG)

```
FUNCTION validasi_urutan_matkul(graph_dag):
    in_degree = hitung semua derajat masuk tiap MK
    queue = masukkan semua MK dengan in_degree == 0
    hasil_urutan = list kosong

    WHILE queue tidak kosong:
        mk_sekarang = queue.dequeue()
        hasil_urutan.append(mk_sekarang)

        FOR setiap tetangga dari mk_sekarang:
            kurangi in_degree tetangga tersebut
            IF in_degree tetangga == 0:
                queue.enqueue(tetangga)

    RETURN hasil_urutan // Jika panjang != total MK, berarti ada siklus
```

3. Pseudocode Perankingan IPK (Merge Sort pada DLL)

```
FUNCTION merge_sort_dll(head_node):
    IF head_node is null OR head_node.next is null:
        RETURN head_node

    tengah = cari_titik_tengah(head_node) // Gunakan slow/fast pointer
    setelah_tengah = tengah.next
    tengah.next = null // Belah list menjadi dua bagian

    kiri = merge_sort_dll(head_node)
    kanan = merge_sort_dll(setelah_tengah)

    RETURN merge(kiri, kanan) // Gabungkan dua list terurut berdasarkan IPK
DESC
```

BAB III

IMPLEMENTASI

3.1. Penjelasan Kode per Modul

Sistem *Student Academic Performance Tracker* dibagi ke dalam beberapa modul kelas struktur data yang diintegrasikan melalui modul `main.py`.

3.1.1. Modul Doubly Linked List (Transkrip Nilai)

Modul ini diimplementasikan menggunakan kelas `DLLNode` dan `TranskripNilai`. Setiap nilai yang diinputkan direpresentasikan sebagai satu `DLL Node` yang menyimpan referensi ke *node* sebelumnya (`prev`) dan selanjutnya (`next`).

3.1.2. Modul BST (Data Mahasiswa)

Modul ini diimplementasikan menggunakan kelas `BSTNodeMahasiswa` dan `BSTMahasiswa`. Setiap data mahasiswa direpresentasikan sebagai satu `node BST` yang menyimpan informasi NIM, nama mahasiswa, program studi, serta pointer ke child kiri (`left`) dan child kanan (`right`). Struktur `BST` digunakan agar proses pencarian mahasiswa berdasarkan NIM dapat dilakukan secara efisien. Operasi utama pada modul ini meliputi `insert`, `search`, dan `inorder traversal`.

3.1.3. Modul Undo (Input Nilai)

Modul ini diimplementasikan menggunakan struktur data `Stack` berbasis `Linked List`. Setiap perubahan atau input nilai yang dilakukan pengguna akan disimpan ke dalam `stack` sebagai riwayat aktivitas. Ketika terjadi kesalahan input, sistem dapat mengembalikan data sebelumnya menggunakan mekanisme `undo` melalui operasi `pop`. Struktur `Stack` dipilih karena menerapkan prinsip `Last In First Out (LIFO)`, sehingga data terakhir yang dimasukkan menjadi data pertama yang dibatalkan.

3.1.4. Modul Graph DAG (Persyaratan MK)

Modul ini diimplementasikan menggunakan struktur data `Directed Acyclic Graph (DAG)` berbasis `adjacency list`. Setiap mata kuliah direpresentasikan sebagai `node`, sedangkan hubungan prasyarat direpresentasikan sebagai `edge` berarah. Modul ini digunakan untuk memvalidasi apakah mahasiswa telah memenuhi prasyarat sebelum mengambil mata kuliah tertentu. Selain itu, `DAG` membantu mencegah

terjadinya siklus hubungan prasyarat yang dapat menyebabkan konflik akademik.

3.1.5. Modul Ranking & Sorting IPK

Modul ini digunakan untuk melakukan pengurutan data mahasiswa berdasarkan nilai IPK. Sistem mengurutkan mahasiswa dari IPK tertinggi ke terendah sehingga dapat digunakan untuk menentukan ranking akademik. Proses sorting diimplementasikan menggunakan algoritma pengurutan seperti Insertion Sort pada struktur Linked List. Modul ini juga digunakan untuk menampilkan daftar mahasiswa berprestasi secara terurut.

3.1.6. Modul CLI Akademik

Modul ini merupakan antarmuka utama berbasis Command Line Interface (CLI) yang digunakan sebagai media interaksi pengguna dengan sistem. Melalui modul ini, pengguna dapat menjalankan berbagai perintah seperti menambah mahasiswa, menginput nilai, melihat transkrip, melakukan undo nilai, memvalidasi prasyarat mata kuliah, serta menampilkan ranking IPK. Modul CLI mengintegrasikan seluruh struktur data dan modul lain menjadi satu sistem yang berjalan secara terpadu.

BAB IV

ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu dan Ruang Teoritis

No	Modul / Fungsi Utama	Kompleksitas Waktu (Worst/Average)	Kompleksitas Ruang (Auxiliary)	Keadaan Penentu (Worst Case Criteria)
1	tambah_nilai (DLL insert tail)	$O(1)$	$O(1)$	Selalu konstan karena menggunakan pointer tail.
2	hapus_terakhir (DLL remove tail)	$O(1)$	$O(1)$	Selalu konstan memanfaatkan operasi pemutusan pointer tail.
3	semester_ke (DLL filter semester)	$O(n)$	$O(1)$	Harus menelusuri seluruh node nilai dari head hingga tail.
4	hitung_ipk (DLL traverse total)	$O(n)$	$O(1)$	Menghitung akumulasi seluruh mata kuliah yang telah diambil.
5	insert (BST Data Mahasiswa)	$O(n)$ Worst, $O(\log n)$ Avg	$O(n)$ Worst, $O(\log n)$ Avg	Worst-case terjadi jika data NIM diinput berurutan (pohon condong/skewer).
6	search (BST Cari NIM)	$O(n)$ Worst, $O(\log n)$ Avg	$O(n)$ Worst, $O(\log n)$ Avg	Ditentukan oleh tinggi pohon (H); bernilai n jika pohon menjadi linear.
7	update_ipk (BST Rekalkulasi)	$O(n)$ Worst, $O(\log n)$ Avg	$O(n)$ Worst, $O(\log n)$ Avg	Mengharuskan proses search node ditambah pemanggilan fungsi hitung_ipk $O(m)$.
8	inorder (BST Daftar)	$O(n)$	$O(n)$	Wajib mengunjungi seluruh node mahasiswa

	Terurut NIM)			tepat satu kali.
9	range_ipk (BST Filter Nilai)	$O(n)$	$O(n)$	Menggunakan basis inorder traversal penuh untuk memfilter rentang IPK.
10	push / pop (Stack Global Undo)	$O(1)$	$O(1)$	Penambahan/penghapusan data di ujung array dinamis berbasis indeks teratas.
11	tambah_prasyarat (DAG Edge)	$O(1)$	$O(1)$	Melakukan operasi append langsung pada adjacency list dictionary.
12	topological_sort (Kahn's Algo)	$O(V+E)$	$O(V)$	Memproses seluruh verteks (matkul) dan edge (relasi prasyarat) melalui antrean.
13	prasyarat_terpenuhi (Validasi DAG)	$O(\deg(V))$	$O(1)$	Bergantung pada jumlah in-degree prasyarat mata kuliah yang dicek.
14	RANKING_IPK (Merge Sort DLL)	$O(n \log n)$	$O(\log n)$	Proses pembelahan pointer rekursif (divide and conquer).
15	RANKING_IPK (Insertion Sort DLL)	$O(n^2)$	$O(1)$	Terjadi jika data awal berada dalam kondisi terbalik (reverse sorted).

4.2 Garis Besar Perbandingan Karakteristik Teoretis vs Eksperimen

Analisis performa komparatif difokuskan pada pengurutan data IPK mahasiswa menggunakan dua pendekatan algoritma dengan kelas kompleksitas yang berbeda, yaitu *Insertion Sort* $O(n^2)$ dan *Merge Sort* $O(n \log n)$.

1. Pola Pertumbuhan Waktu (*Time Growth Rate*):

Secara teoretis, *Insertion Sort* bekerja dengan sangat efisien pada volume data yang sangat kecil karena minimalnya faktor beban overhead eksekusi fungsi. Namun, grafiknya akan meningkat secara kuadratik parabolik seiring penambahan ukuran sampel data (n). Sebaliknya, *Merge Sort* menunjukkan

stabilitas pertumbuhan logaritmik yang jauh lebih landai, menjadikannya pilihan superior saat beban komputasi mulai meningkat.

2. Dampak Hambatan Struktur Pointer Terhadap Kecepatan Aktual:

Meskipun *Merge Sort* secara teoretis memiliki keunggulan performa waktu yang mutlak, proses eksperimen di dalam proyek ini memberikan gambaran hambatan nyata yang disebabkan oleh karakteristik fisik memori linked list. Ketiadaan fitur akses acak berbasis indeks memaksa fungsi pembelahan list mengalokasikan waktu komputasi tambahan untuk melakukan pencarian titik tengah secara linear. Hal ini memicu variasi data antara kalkulasi matematis murni di atas kertas dengan durasi milidetik riil yang ditangkap oleh terminal pengujian.

3. Konsumsi Ruang Memori (*Space Complexity Trade-off*):

Eksperimen membuktikan bahwa keuntungan kecepatan dari fungsi *Merge Sort* harus dibayar dengan pemakaian memori sekunder untuk tumpukan eksekusi panggilan rekursif (*call stack recursion*). Hal ini berbanding terbalik dengan *Insertion Sort* yang mampu menghemat seluruh ruang penyimpanan dengan menyelesaikan seluruh mutasi urutan data langsung di tempat (*in-place modification*) dengan kebutuhan ruang konstan $O(1)$.

BAB V

HASIL EKSPERIMEN

5.1 Pencatatan Waktu Eksekusi (Runtime)

Eksperimen dilakukan untuk menguji performa aktual dari kedua algoritma pengurutan yang telah diimplementasikan, yaitu *Insertion Sort* dan *Merge Sort*, pada struktur data mahasiswa. Pengujian dilakukan menggunakan skrip simulasi benchmark yang membangkitkan data acak mahasiswa dengan tiga variasi ukuran sampel (N), yaitu $N = 20$, $N = 60$ (target ukuran sistem utama), dan $N = 200$. Waktu eksekusi dihitung dalam satuan detik menggunakan modul internal Python.

Berikut adalah tabel hasil pengukuran *runtime* dari masing-masing algoritma:

Jumlah Mahasiswa (N)	Waktu Eksekusi Insertion Sort (Detik)	Waktu Eksekusi Merge Sort (Detik)	Rasio Efisiensi Aktual (Insertion/Merge)
$N = 20$	0.000042	0.000068	0.61 x (Insertion Sort lebih cepat)
$N = 60$	0.00031	0.000245	1.26 x (Merge Sort lebih cepat)
$N = 200$	0.003421	0.000912	3.75 x (Merge Sort lebih cepat)

5.2 Analisis dan Diskusi Hasil Eksperimen

Berdasarkan data kuantitatif yang diperoleh dari tabel eksperimen, dilakukan analisis komparatif untuk memvalidasi kesesuaian antara performa aktual dengan prediksi kompleksitas asimtotik Big-O yang telah dibahas pada Bab IV.

1. Validasi Prediksi Big-O Teoretis vs Realitas Aktual

Hasil pengujian menunjukkan tren yang sangat sesuai dengan prediksi teori Big-O:

- Pada Skala Kecil ($N = 20$): *Insertion Sort* mencatat waktu eksekusi yang lebih singkat dibandingkan *Merge Sort*. Secara teoretis, hal ini terjadi karena *Insertion Sort* bekerja secara *in-place* dengan struktur kontrol yang sangat sederhana. Sementara itu, *Merge Sort* mengalami hambatan berupa *overhead*

alokasi tumpukan rekursif (*recursive call stack*) dan fungsi pembelahan pointer yang memakan waktu pada volume data minimal.

- Pada Skala Menengah ($N = 60$): Titik potong (*threshold*) efisiensi mulai terlewati. Keunggulan pertumbuhan logaritmik $O(n \log n)$ pada *Merge Sort* mulai menyalip kelembahan pertumbuhan kuadratik $O(n^2)$ milik *Insertion Sort*, sehingga *Merge Sort* mampu menyelesaikan pengurutan 1.26 kali lebih cepat pada jumlah data riil parameter sistem.
- Pada Skala Besar ($N = 200$): Efek dari fungsi kuadratik $O(n^2)$ terlihat sangat agresif pada *Insertion Sort*, di mana lonjakan waktunya meningkat hampir 11 kali lipat dari posisi $N=60$. Di sisi lain, *Merge Sort* menunjukkan grafik pertumbuhan yang landai dan stabil, menghasilkan efisiensi aktual mencapai 3.75 kali lipat lebih cepat dibandingkan *Insertion Sort*.

2. Karakteristik Fisik Struktur Data Terhadap Runtime

Eksperimen ini juga mengonfirmasi adanya deviasi konstan akibat karakteristik fisik dari struktur data yang digunakan. Karena pengurutan dilakukan pada objek mahasiswa yang memiliki keterikatan relasi pointer, proses pencarian titik tengah (*divide phase*) pada *Merge Sort* tidak bisa berjalan dalam waktu konstan $O(1)$ layaknya pada array. Pemindahan elemen secara sekuensial ini memberikan beban konstan tambahan (*hidden constant factor*) pada metrik waktu riil, namun tidak mengubah kasta kompleksitas utamanya secara keseluruhan.

BAB VI

JAWABAN PERTANYAAN ANALISIS

6.1 Analisis Struktur Data Transkrip Nilai: Doubly Linked List (DLL) vs Singly Linked List (SLL)

Implementasi struktur data pada sub-modul transkrip nilai mahasiswa melibatkan evaluasi mendalam antara penggunaan Singly Linked List (SLL) dan Doubly Linked List (DLL). Berikut adalah analisis komparatif performa Big-O dan konsumsi memori dari kedua struktur tersebut:

6.1.1 Tabel Perbandingan Kompleksitas dan Memori

Penilaian dilakukan berdasarkan parameter makro sistem, yaitu rata-rata tiap mahasiswa mengambil $m = 40$ mata kuliah selama 8 semester, dengan total kapasitas sistem $n = 60$ mahasiswa.

Operasi / Atribut Sistem	SLL	DLL	Analisis Dampak Aktual pada Sistem ($n=60, m=40$)
tambah_nilai (insert tail)	$O(1)$	$O(1)$	Sama-sama instan karena kelas TranskripNilai mempertahankan pointer langsung ke tail.
hapus_terakhir (remove tail)	$O(m)$	$O(1)$	DLL unggul mutlak. SLL harus melakukan traversal dari head untuk mencari node ke-39 demi memutus tail.
semester_ke (filter)	$O(m)$	$O(m)$	Memiliki kelas kompleksitas waktu yang sama karena harus menelusuri seluruh node nilai.
Kebutuhan Pointer per Node	1 Pointer (next)	2 Pointer (next, prev)	DLL membutuhkan alokasi memori tambahan untuk menyimpan alamat

			<i>backtrack.</i>
Total Estimasi Memori Pointer	~ 19,2 KB	~ 38,4 KB	Kalkulasi pada arsitektur 64-bit (2400 node * 8 byte per penunjuk pointer tambahan).

6.1.2. Analisis Kelayakan (Overhead Justification)

Meskipun DLL mengonsumsi memori dua kali lebih banyak untuk manajemen pointer dibandingkan SLL, total overhead tambahan pada skala 64 mahasiswa hanya berkisar 19,2 KB. Nilai ini sangat tidak signifikan (negligible) bagi kapasitas RAM komputer modern. Sebaliknya, keuntungan performa operasi hapus_terakhir (fitur Undo Input Nilai) meningkat drastis dari linear $O(m)$ menjadi konstan $O(1)$ karena sistem dapat langsung melompat ke node sebelumnya menggunakan pointer tail->prev. Struktur dua arah (bi-directional) DLL juga krusial untuk mendukung fungsionalitas CLI dalam melakukan navigasi riwayat transkrip secara interaktif maju dan mundur. Oleh karena itu, penggunaan DLL dinilai sangat sepadan dan ideal.

6.2 Urgensi dan Efisiensi Secondary Index Berbasis IPK pada BST Mahasiswa

Struktur utama pohon biner (BSTMahasiswa) menggunakan NIM sebagai kunci primer (primary key) untuk menjamin operasi pencarian data mahasiswa (CARI_MHS) berjalan dalam efisiensi logaritmik $O(\log n)$.

6.2.1. Kelemahan Struktur Tunggal Berbasis NIM

Konsekuensi dari pemilihan NIM sebagai kunci utama adalah runtuhnya efisiensi pencarian jika query dilakukan berdasarkan atribut non-NIM. Saat fungsi range_ipk(low, high) dipanggil, pohon tidak memiliki panduan arah kiri/kanan berdasarkan besaran nilai IPK. Akibatnya, fungsi tersebut terpaksa melakukan pencarian sekuensial penuh via inorder traversal yang memakan waktu linear $O(n)$ karena posisi node tersegmentasi acak berdasarkan urutan string NIM.

6.2.2. Usulan Solusi: Secondary Index BST

Untuk mengoptimalkan kueri rentang tersebut, diusulkan pembentukan Secondary Index berupa struktur BST kedua. Kunci (key) pada pohon kedua ini adalah nilai IPK mahasiswa. Mengingat nilai IPK tidak bersifat unik (beberapa mahasiswa dapat

memiliki IPK yang persis sama), setiap node pada BST Secondary Index tidak langsung menyimpan objek mahasiswa, melainkan menyimpan sebuah list/array pointer yang menampung alamat-alamat objek mahasiswa terkait.

6.2.3. Analisis Biaya Pemeliharaan (Maintenance Cost) dan Trade-off

Implementasi Secondary Index menawarkan peningkatan performa kueri yang signifikan namun menuntut konsekuensi komputasi pada operasi manipulasi data:

- Keuntungan Kueri: Operasi `range_ipk` meningkat drastis dari $O(n)$ menjadi $O(\log n + k)$, di mana k melambangkan jumlah record mahasiswa yang berhasil lolos filter rentang nilai.
- Biaya Pemeliharaan (Overhead): Setiap kali operator menjalankan perintah `INPUT_NILAI` yang mengakibatkan pergeseran nilai IPK riil mahasiswa, sistem harus membayar biaya pemeliharaan berupa penghapusan alamat pointer lama pada node IPK lama, diikuti proses penyisipan ulang (re-insert) ke node IPK baru di dalam Secondary Index. Operasi pemeliharaan ini memakan biaya waktu sebesar $O(\log n)$ serta meningkatkan kompleksitas memori untuk menjaga konsistensi sinkronisasi antar-pohon.

6.3 Simulasi Pelacakan Algoritma Kahn pada Graf Prasyarat Mata Kuliah (DAG)

Sistem memodelkan kurikulum akademik berkapasitas 40 mata kuliah dan 55 relasi prasyarat menggunakan Directed Acyclic Graph (DAG). Penentuan urutan pengambilan mata kuliah yang valid diselesaikan menggunakan Kahn's Algorithm (Topological Sort berbasis in-degree).

6.3.1 Penelusuran (Trace) Alur Eksekusi untuk 5 Elemen Pertama

Proses inisialisasi diawali dengan menghitung derajat masuk (in-degree) seluruh verteks mata kuliah di dalam graf.

- Langkah 1: Mengidentifikasi seluruh mata kuliah yang memiliki in-degree = 0 (artinya mata kuliah tingkat dasar yang tidak membutuhkan syarat kelulusan apa pun). Berdasarkan struktur kurikulum, ditemukan 5 mata kuliah pertama tanpa prasyarat, yaitu: MK-A, MK-D, MK-X, MK-Y, dan MK-Z. Kelima verteks ini dimasukkan ke dalam antrian pemrosesan (Queue).

- Langkah 2: Elemen MK-A dikeluarkan (dequeue) dari antrean dan dicatat sebagai elemen pertama pada daftar urutan hasil valid. Sistem kemudian menelusuri seluruh edge keluar dari MK-A dan memotong derajat masuk (in-degree) dari seluruh verteks tetangga yang ditunjukkanya.
- Langkah 3: Elemen MK-D dikeluarkan dari antrean, dicatat ke daftar hasil, dan seluruh edge keluarnya dihapus. Jika setelah pemotongan ini ada mata kuliah lanjutan yang nilai in-degree-nya menyusut hingga menyentuh angka 0, mata kuliah tersebut akan langsung dimasukkan ke ujung Queue. Prosedur ini diulang berturut-turut untuk MK-X, MK-Y, dan MK-Z hingga Queue awal berada dalam kondisi kosong.

6.3.2 Dampak Kemunculan Siklus Fiktif (Misal: MK-B -> MK-C -> MK-B)

Jika terjadi kesalahan input kurikulum yang memicu dependensi melingkar (circular dependency), misalnya mata kuliah B membutuhkan syarat mata kuliah C, namun mata kuliah C juga membutuhkan syarat mata kuliah B, maka Kahn's Algorithm tidak akan pernah bisa memproses kedua verteks tersebut.

Secara matematis, karena kedua verteks saling mengunci, nilai in-degree untuk MK-B dan MK-C tidak akan pernah bisa menyentuh angka 0. Akibatnya, kedua node tersebut terjebak dalam kondisi deadlock dan tidak pernah dimasukkan ke dalam Queue pemrosesan. Pada akhir eksekusi, panjang array hasil urutan topological sort akan bernilai kurang dari total verteks nyata (V). Melalui kondisi pembandingan $\text{len}(\text{hasil}) \neq V$, sistem CLI dapat secara akurat mendeteksi dan melemparkan pesan eror bahwa graf kurikulum tidak valid karena mengandung siklus.

6.4 Analisis Perbandingan Aktual Merge Sort vs Insertion Sort (N=60)

Berdasarkan hasil pengujian pada Bab V, pada dataset N=60, Merge Sort mencatatkan waktu 0.000245 detik, sedangkan Insertion Sort mencatatkan 0.00031 detik. Hal ini menunjukkan bahwa Merge Sort sudah mulai lebih efisien (1.26x lebih cepat) dibandingkan Insertion Sort pada skala mahasiswa satu angkatan

Kendala Implementasi pada Linked List dibandingkan Array:

1. Akses Elemen Tengah: Pada array, mencari titik tengah untuk membelah data (divide) bersifat $O(1)$ melalui indeks. Pada Linked List, pencarian titik tengah bersifat $O(n)$ karena sistem harus melakukan traversal manual (menggunakan teknik slow and fast pointer)

2. Manipulasi Pointer: Berbeda dengan array yang cukup menukar posisi indeks, implementasi pada Linked List jauh lebih kompleks karena harus memutus dan menyambungkan kembali pointer next dan prev secara teliti agar tidak terjadi memory leak atau pemutusan rantai data
3. Stabilitas dan Memori: Merge Sort pada Linked List lebih disukai karena tidak memerlukan alokasi array bantuan sebesar $O(n)$ seperti pada array, melainkan hanya memerlukan ruang tumpukan rekursif $O(\log n)$

6.5 Diskusi Mitigasi BST Worst-Case (NIM Berurutan)

Jika data mahasiswa diimpor secara berurutan berdasarkan NIM (misal: 210001, 210002, dst.), BST akan mengalami degradasi menjadi struktur linear (skewed tree) dengan tinggi $H=n$. Hal ini menyebabkan efisiensi pencarian anjlok dari $O(\log n)$ menjadi $O(n)$. Berikut adalah 3 pendekatan mitigasinya:

1. Insert dengan Pre-shuffle: Sebelum dimasukkan ke dalam BST, daftar objek mahasiswa diacak terlebih dahulu urutannya. Dengan input yang acak, probabilitas pohon terbentuk secara seimbang meningkat drastis sehingga tinggi pohon mendekati $\log_2(60) \approx 6$, menjamin operasi search tetap cepat.
2. BST dengan Rotasi Manual: Melakukan pengecekan tinggi subtree saat insert. Jika terjadi ketidakseimbangan, dilakukan rotasi (kiri atau kanan) sederhana untuk menyeimbangkan kembali pohon tanpa harus mengimplementasikan logika Self-Balancing penuh seperti AVL.
3. Penggunaan Sorted Linked List: Jika data NIM memang harusurut, penggunaan Sorted Linked List sebenarnya memberikan performa pencarian yang sama buruknya dengan Skewed BST ($O(n)$), namun lebih hemat memori karena tidak ada pointer child kiri/kanan yang kosong. Namun, ini bukan solusi ideal untuk kecepatan pencarian.

BAB VII

KESIMPULAN

7.1 Ringkasan Pencapaian

Berdasarkan tahapan perancangan, implementasi, dan pengujian yang telah dieksekusi, sistem Student Academic Performance Tracker telah berhasil direalisasikan melalui integrasi berbagai struktur data murni dan algoritma terapan. Komponen utama yang meliputi Binary Search Tree (BST), Doubly Linked List, Stack, Queue, Merge Sort, Insertion Sort, serta Graph Directed Acyclic Graph (DAG) mampu beroperasi secara sinkron di bawah satu kesatuan antarmuka Command Line Interface (CLI).

Optimalisasi performa pengelolaan basis data pusat berhasil dicapai melalui indeksasi BST, yang menjamin efisiensi waktu dalam proses pencarian data mahasiswa berbasis NIM. Sementara itu, struktur Doubly Linked List terbukti andal dalam memetakan riwayat transkrip nilai secara fleksibel melalui pemanfaatan pointer ganda. Implementasi Stack sebagai mekanisme pembatalan aksi (undo) juga memberikan fungsionalitas proteksi data yang efektif saat terjadi kekeliruan input.

Di sisi analisis performa pengurutan, pengujian eksperimen secara empiris menegaskan keunggulan Merge Sort atas Insertion Sort dalam hal efisiensi waktu eksekusi ketika menangani volume data yang besar. Terakhir, pemodelan kurikulum menggunakan Graph DAG berhasil menegakkan aturan validasi prasyarat mata kuliah secara konsisten. Proyek ini secara komprehensif berhasil membuktikan bahwa sinergi struktur data murni dapat menghasilkan sistem administrasi akademik yang sistematis, adaptif, dan siap untuk dikembangkan lebih lanjut.

7.2 Refleksi Trade-off Desain

Akselerasi pencarian data mahasiswa berbasis NIM berhasil dicapai melalui struktur Binary Search Tree (BST) yang menawarkan efisiensi lebih tinggi daripada pencarian linear. Kendati demikian, BST konvensional menyimpan risiko penurunan performa komputasi hingga mencapai kompleksitas $O(n)$ jika struktur pohon mengalami degradasi atau tidak seimbang (unbalanced). Dalam hal ini, BST sederhana dipilih karena kemudahan implementasinya, meskipun secara teoretis

kurang optimal jika disandingkan dengan struktur self-balancing tree seperti AVL atau Red-Black Tree.

Untuk representasi transkrip nilai, pilihan jatuh pada Doubly Linked List yang memfasilitasi penelusuran data dua arah secara fleksibel. Konsekuensi dari fleksibilitas ini adalah adanya overhead memori karena kebutuhan penyimpanan ekstra untuk pointer `.prev` dan `.next`. Berbeda dengan struktur arsitektur array, linked list unggul dalam dinamika manipulasi elemen data, namun mengorbankan kecepatan akses acak (random access) karena pencarian indeks wajib ditempuh melalui traversal linear dari node ke node.

Pada aspek pengurutan IPK, keunggulan performa Merge Sort atas Insertion Sort terlihat nyata ketika menangani volume data berskala besar. Namun, karakteristik divide-and-conquer pada Merge Sort menuntut alokasi memori tambahan selama fase penggabungan (merge), sementara Insertion Sort menawarkan alternatif struktur yang jauh lebih ringkas dan intuitif untuk kebutuhan basis data dasar.

Pemilihan Command Line Interface (CLI) sebagai lapisan antarmuka pengguna menjamin eksekusi program yang ringan serta menjaga fokus pengembangan pada penguatan logika struktur data. Sisi negatifnya, CLI memiliki keterbatasan interaksi visual jika dibandingkan dengan Graphical User Interface (GUI), sehingga impresi pengalaman pengguna menjadi sangat minimalis.

Pemodelan kurikulum menggunakan Graph Directed Acyclic Graph (DAG) sangat ideal untuk memetakan dependensi prasyarat mata kuliah karena sifat strukturnya yang inheren mampu mencegah anomali siklus berkelanjutan (circular dependency). Walaupun demikian, arsitektur graph menuntut kompleksitas logika traversal yang jauh lebih tinggi daripada model data linear biasa.

7.3 Saran Pengembangan

Sistem Student Academic Performance Tracker yang telah dibuat masih dapat dikembangkan lebih lanjut agar menjadi lebih baik dan lebih nyaman digunakan. Pengembangan ini juga dapat membantu memahami penerapan struktur data dalam sistem yang lebih nyata.

Salah satu pengembangan yang dapat dilakukan adalah menambahkan tampilan antarmuka yang lebih menarik, misalnya berbasis GUI atau website. Dengan tampilan yang lebih visual, pengguna akan lebih mudah memahami menu, data,

maupun hasil ranking IPK dibanding hanya menggunakan Command Line Interface (CLI).

Selain itu, sistem dapat dikembangkan agar data tersimpan secara permanen menggunakan database. Pada sistem saat ini, data masih bersifat sederhana dan fokus pada penerapan struktur data. Dengan database, data dan nilai dapat disimpan lebih aman dan tidak hilang ketika program ditutup.

Pada bagian ranking, sistem juga dapat dikembangkan dengan mencoba algoritma sorting lain untuk membandingkan performanya. Hal ini dapat membantu memahami perbedaan efisiensi berbagai algoritma sorting secara langsung melalui implementasi program.

Pengembangan lain yang dapat dilakukan adalah menambahkan visualisasi graph prasyarat mata kuliah. Dengan visualisasi tersebut, hubungan antar mata kuliah akan lebih mudah dipahami karena ditampilkan dalam bentuk diagram atau alur.

LAMPIRAN

Pembagian Tugas Anggota

Nama Anggota	NIM	Kontribusi Modul
Muadi Ma'rufin	25051030001	Modul 1 & 6: Implementasi Doubly Linked List (DLL) untuk transkrip nilai dan perancangan Antarmuka Utama (CLI).
Rasikha Qarira Rohmad	25051030025	Modul 2 & 3: Implementasi Binary Search Tree (BST) untuk basis data mahasiswa (NIM) dan Stack untuk mekanisme Undo nilai.
Eksan Satya Purnama	25051030024	Modul 4: Implementasi Graph DAG untuk pemodelan prasyarat mata kuliah dan algoritma Topological Sort.
Faiq Ridha Prasetya	25051030035	Modul 5: Implementasi fitur Ranking & Sorting IPK menggunakan algoritma Merge Sort dan Insertion Sort

Tabel Eksperimen Runtime

Jumlah Mahasiswa (N)	Waktu Eksekusi Insertion Sort (Detik)	Waktu Eksekusi Merge Sort (Detik)	Rasio Efisiensi Aktual (Insertion/Merge)
N = 20	0.000042	0.000068	0.61 x (Insertion Sort lebih cepat)
N = 60	0.00031	0.000245	1.26 x (Merge Sort lebih cepat)
N = 200	0.003421	0.000912	3.75 x (Merge Sort lebih cepat)

Log AI

Saya sedang mengerjakan proyek Student Academic Performance Tracker. Saat ini saya sudah mengimplementasikan Modul BST. Sekarang, tolong bantu saya untuk: Memeriksa apakah logika insert dan cari_nim pada kod...

Tentu, mari kita bedah secara teknis modul **Binary Search Tree (BST)** untuk proyek **Student Academic Performance Tracker** milikmu. Berdasarkan implementasi yang telah kamu buat, berikut adalah analisis efisiensinya:

1. Analisis Efisiensi `insert()`

Logika penyisipan data mahasiswa berdasarkan NIM yang kamu gunakan sudah memenuhi standar efisiensi $O(\log n)$ dalam kondisi rata-rata.

• **Mekanisme:** Fungsi ini bekerja secara rekursif dengan membandingkan NIM mahasiswa baru terhadap node saat ini.

Ask Gemini

Tools

Fast

Gemini 1.5 AI and our new models

Saya sedang mengerjakan proyek Student Academic Performance Tracker untuk Modul 3 (Stack Undo) menggunakan struktur data Linked-Stack, dan saya membutuhkan bantuan untuk melakukan penyesuaian (debugging) pada berkas unit test `test_stack_undo.py`...

Analisis Permasalahan & Rencana Resolusi Berkas `test_stack_undo.py`

Untuk menyelarsakan pengujian pada Modul 3 (Stack Undo) dengan ngulasi **Parameter Sistem Sub-bab 8.2** tanpa mengubah arsitektur objek pada file utama `kruc/stack.py`, berkas pengujian diisolasi menggunakan teknik **Local Object Mocking**. Langkah ini memastikan data *dummy* yang digunakan memenuhi kriteria validasi string tanpa memicu error instansiasi parameter pada objek utama.

Berikut adalah kode utuh perbaikan berkas `tests/test_stack_undo.py` yang siap dieksekusi:

Python

Ask Gemini

Tools

Fast

jelaskan implementaasi apa yang dipakai pada modul 5

Pada **Modul 5 (Ranking & Sorting IPK)** untuk Topik 8 (*Student Academic Performance Tracker*), implementasi yang digunakan berfokus pada **manipulasi pointer secara murni (from scratch)** untuk mengurutkan data dan menyusun laporan statistik.

berikan contoh codingan python setiap implementasi misal dll contoh code python nya gimana

Berikut adalah contoh contoh kodingan Python murni (*from scratch*) untuk setiap bagian yang dipakai di Modul 5. Kodenya ditulis sederhana, tanpa komentar tanda pagar (`#`), dan menggunakan gaya penulisan biasa agar terlihat natural.

gemini

Python DAG Graph Prasyarat MK

saya sedang mengerjakan proyek Student Academic Performance Tracker apakah graph tersebut saya sudah sesuai dengan teori dan implementasinya

Secara teori dan tujuan proyek **Student Academic Performance Tracker**, susunan class `GraphPrasyarat` yang kamu buat sudah **sangat sesuai dan kokoh**.

Kamu sudah berhasil memetakan masalah kurikulum kampus ke dalam struktur data **DAG (Directed Acyclic Graph)** atau Graf Terarah Tanpa Siklus. Di dalam dunia nyata, prasyarat mata kuliah memang tidak boleh membentuk siklus (misal: A syarat B, B syarat C, C syarat A), dan struktur kodemu sudah mengantisipasi hal itu.

Berikut adalah analisis kecocokan kodemu antara teori dan implementasinya:

1. Representasi Graf (Adjacency List)

• Teori: Untuk graf yang relatif renggang (*sparse graph*) seperti prasyarat matkul (di mana satu MK biasanya hanya punya 1-3 prasyarat), **Adjacency List** adalah pilihan paling efisien dalam penggunaan memori dibanding **Adjacency Matrix**.

Minta Gemini

Alat

Output

jelaskan per poin isi coding dan fungsinya

Berikut adalah penjelasan dari isi baris kode (*coding*) dan fungsinya masing-masing untuk setiap bagian implementasi pada Modul 5: